RFC-2024-001: /ship-it

# RFC-2024-001: /ship-it — Guided Delivery Skill for Developer Self-Service Deployment

| Field | Value |
| --- | --- |
| **RFC** | RFC-2024-001 |
| **Title** | /ship-it — Guided Delivery Skill for Developer Self-Service Deployment |
| **Author** | DevOps Engineering |
| **Status** | DRAFT — Open for Comment |
| **Created** | 2026-03-09 |
| **Repository** | https://github.com/SleepNumberInc/sleep-number-claude-code-plugins Connect your Github account |
| **Audience** | DevOps Engineers, Platform/Infrastructure Engineers |

---

Table of Contents


1. Problem Statement
2. Proposed Solution
3. Process Flow — User Day-to-Day
4. Where DevOps and GitHub Actions Get Involved
5. GitHub Actions Architecture
6. The .ship-it.yml Configuration (DevOps-Owned)
7. Prerequisites
8. End-to-End Workflow: Code to Production
9. Security Considerations
10. Recommendation: Containers as the Standardized Deployment Unit
11. Open Questions for Reviewers
12. Rollout Plan
13. How to Review / Comment


---

## 1. Problem Statement

New developers and non-DevOps engineers ("vibe coders") struggle to move their working code into production. The current process requires knowledge of:

- Git branching strategies
- CI/CD pipeline configuration (YAML syntax, workflow triggers, environment gates)

- PR conventions (labels, descriptions, review assignment)
- Infrastructure prerequisites (SSL, DNS, RBAC, app registration)
- Which teams to contact and in what order

This creates a bottleneck where developers wait on DevOps to do things for them, or they skip steps and ship without proper checks. Both outcomes are bad.

**What we want instead:** A developer finishes their code, runs one command, and the system handles everything — branching, committing, pipeline setup, PR creation, reviewer assignment, and production prerequisites. DevOps stays in control of the guardrails but is not a bottleneck for every deployment.

---

### 2. Proposed Solution

`/ship-it` — a Claude Code plugin (skill) that a developer runs from their terminal. It automates the entire path from local code to a production-ready pull request with zero questions asked.

The skill is installed via the Claude Code plugin marketplace from:
** https://github.com/sleepnumberinc/ship-it.git** Connect your Github account

#### Scope boundary

`/ship-it` handles the path from local code to a production-ready pull request. It does not build, package, or deploy the application. All build, packaging (container or otherwise), and deployment logic lives in GitHub Actions workflows, which are owned and configured by the DevOps team. The skill makes no assumptions about whether an app is containerized, deployed as a zip, or hosted on any particular platform.

#### What the developer does

```
/ship-it
```

That's it. One command. The skill handles everything else.

#### What the skill does (behind the scenes)

1. Detects the repo, branch, auth status, and project type
2. Reads the DevOps-managed `.ship-it.yml` config from the repo
3. Creates a branch, commits changes, pushes
4. Generates a caller workflow that references the org's shared reusable workflow (if no workflow exists yet)
5. Creates a pull request with the correct labels, reviewers, description, and go-live checklist
6. Reports back to the developer: "Done! The team will let you know when it's live."

#### Two modes

| Command | What it does |
|---|---|
| `/ship-it` | Ship to production. Creates a PR, assigns reviewers, full safety checks. |
| `/ship-it save` | Save work in progress. Commits, pushes, creates a draft PR. No review, no deployment. |

### 3. Process Flow — User Day-to-Day

This section describes the complete lifecycle of a code change from the developer's perspective, and where automation, GitHub Actions, and DevOps involvement occur at each stage.

```
DEVELOPER'S DAY                        WHAT HAPPENS BEHIND THE SCENES
===========================
==========================================

1. Developer writes code               (nothing — local work only)
       |
       v
2. Developer runs /ship-it save        Skill: commit + push + draft PR
   (optional, can repeat)              GHA: nothing runs (draft PRs
don't trigger)
       |                               DevOps: not involved
       v
3. Developer runs /ship-it             Skill: branch + commit + push +
PR + labels
       |                                      + reviewers + go-live
checklist
       |                               GHA: build-and-validate runs on
PR
       |                               DevOps: not involved yet
       v
4. PR is open, checks run              GHA: build, lint, security scan
execute
       |                               DevOps: auto-notified as
reviewer
       |                                       (from .ship-it.yml or
CODEOWNERS)
       v
5. Code review                         Peer developer or tech lead
reviews code
       |                               DevOps reviews infra/deploy
concerns only
       v
6. PR is merged to main                GHA: deploy-dev job runs
automatically
       |
       v
7. Dev deployment succeeds             Developer: validates app in dev
environment
       |                               Developer: confirms it works as
expected
       |                               (if broken, developer fixes and
re-runs /ship-it)
       v
8. Developer signs off on dev          DevOps: reviews dev validation,
approves prod
       |                               GHA: deploy-prod job runs
       v
9. Production deployment               GHA: deploys to production
       |                               Owning team monitors (see note
below)
       v
10. Developer is notified              "Your app is live!"
```

#### Feedback loop

If something breaks at any stage, the developer is brought back in:

```
PR checks fail ———> Developer fixes code, runs /ship-it again
(updates PR)
Code review feedback —> Developer addresses comments, runs /ship-it
again
Dev validation fails —> Developer fixes code, runs /ship-it again (new
PR cycle)
Prod issue ————————> Developer fixes code, runs /ship-it again
(hotfix path)
```

The developer is never blocked waiting on DevOps to diagnose an issue. They own the fix, `/ship-it` handles getting it back through the pipeline.

#### Where each role is involved

| Stage | Developer | /ship-it Skill | GitHub Actions | DevOps Engineer |
|---|---|---|---|---|
| Write code | **Active** | — | — | — |
| Save progress | Runs `/ship-it save` | Commits, pushes, draft PR | — | — |
| Ship to prod | Runs `/ship-it` | Branch, PR, labels, reviewers | — | — |
| PR checks | Monitors results | — | **Runs build/lint/scan** | — |
| Code review | Addresses feedback | — | — | Reviews infra concerns |
| Merge | — | — | Auto-triggers deploy | Peer/tech lead approves PR |
| Dev deploy | — | — | **Deploys to dev** | — |
| Dev validation | **Validates app in dev** | — | Waits for approval | Available if needed |
| Prod approval | Signs off on dev | — | Waiting | **Approves prod deploy** |
| Prod deploy | — | — | **Deploys to prod** | — |
| Live | **Notified** | — | — | — |

#### Review model: avoiding the DevOps bottleneck

To prevent DevOps from becoming the bottleneck for every PR:

| Review type | Who reviews | When DevOps is needed |
|---|---|---|
| **Code review** | Peer developer or tech lead | Only if changes touch infra, workflows, or `.ship-it.yml` |
| **Dev validation** | Developer (owns their app) | DevOps available for infra issues, not app-level testing |

| | | |
|---|---|---|
| **Prod approval** | DevOps (environment gate) | Yes — this is the one gate DevOps owns |

**Key insight:** The developer is active throughout the lifecycle — writing code, addressing review feedback, and validating in dev. DevOps is only a hard gate at production approval, not at every stage. Code review is a team responsibility shared across peers and tech leads, not an exclusive DevOps function.

---

### 4. Where DevOps and GitHub Actions Get Involved

#### DevOps responsibilities (one-time setup per repo)

| Task | When | How |
|---|---|---|
| Drop `.ship-it.yml` into the repo | Once, at repo creation | Copy the template, set reviewers and environments |
| Configure GitHub Environments | Once, at repo creation | Settings > Environments > Add `dev` and `prd`, set required reviewers on `prd` (see reviewer guidance below) |
| Configure deployment credentials | Once, at repo creation | Add secrets/OIDC for the target platform (see credential table below) |
| Configure branch protection on `main` | Once, at repo creation | Require PR, require status checks, require review |
| Review and approve PRs | Ongoing | Normal PR review process |
| Approve production deployments | Ongoing | GitHub Environment approval UI |

##### Deployment credentials by platform

GitHub Environments need credentials to authenticate with the deployment target. The credential type depends on the platform:

| Platform | Credential type | GitHub secrets/config needed | Setup |
|---|---|---|---|
| **Azure (ACA, App Service)** | App Registration + OIDC federation | `AZURE_CLIENT_ID`, `AZURE_TENANT` | Create App Registration, add federated |

| | | | |
|---|---|---|---|
| | | `_ID`, `AZURE_SUBSCRIPTION_ID` | credential for the repo, assign RBAC to the ACA Environment and ACR |
| **AWS (ECS, Lambda, etc.)** | IAM Role + OIDC federation | `AWS_ROLE_ARN`, `AWS_REGION` | Create IAM role with GitHub OIDC trust policy, scope permissions to the target resources |
| **Kubernetes (AKS, EKS, k3s)** | Kubeconfig or service account token | `KUBE_CONFIG` or `KUBE_TOKEN`, `KUBE_SERVER`, `KUBE_NAMESPACE` | Create a service account with deploy permissions, export kubeconfig or token |

> **Recommendation:** Use OIDC federation (passwordless) wherever possible. This avoids storing long-lived secrets in GitHub and allows credential rotation at the identity provider level. For k3s or self-managed Kubernetes clusters that don't support OIDC federation, use a service account token stored as a GitHub Environment secret with appropriate expiration.

> **Note:** Credentials should be scoped per environment. The `dev` environment uses credentials with access to dev resources only; `prd` uses credentials with access to production resources only. Never share credentials across environments.

#### Who should be the reviewers?

There are two distinct reviewer roles in this workflow. They serve different purposes and may be different people:

| Reviewer role | Where it's configured | Who should it be | Purpose |
|---|---|---|---|
| **PR code reviewers** | `.ship-it.yml` `reviewers` field or `CODEOWNERS` | Peer developers, tech leads, DevOps (for infra changes) | Review the code for correctness, quality, and security |
| **Production environment approvers** | GitHub repo Settings > Environments > | DevOps lead, team lead, or on-call engineer | Gate production deployments — confirm dev |

| | | | |
|---|---|---|---|
| | `prd` > Required reviewers | | validation passed and app is safe to go live |

**Guidance for production environment approvers:**

- Should be someone with **operational awareness** — they understand what's running in production and the blast radius of a deployment
- Typically a DevOps engineer, platform engineer, or engineering manager — not the developer who wrote the code
- Add **at least 2 people** to avoid a single point of failure (GitHub requires any 1 of the listed reviewers to approve)
- Consider using a **GitHub team** (e.g., `@sleepnumberinc/devops`) rather than individual users, so coverage doesn't depend on one person's availability
- The `dev` environment should **not** require approval — it deploys automatically on merge so the developer can validate quickly

> **Open question for reviewers:** Should production environment approvers be standardized across all repos (e.g., the DevOps team for everything), or should each team designate their own approvers? A centralized model is simpler but could become a bottleneck; a distributed model scales better but requires each team to maintain their approver list.

**DevOps responsibilities (ongoing)**

| Task | Frequency | Trigger |
|---|---|---|
| Review PRs labeled `intent:prod-ready` | Per PR | Auto-assigned via `.ship-it.yml` reviewers |
| Approve production environment gate | Per deployment | GitHub notifies when deploy-dev succeeds |
| Update `.ship-it.yml` if team/process changes | As needed | Manual edit, committed to repo |
| Maintain shared reusable workflow | As needed | Pipeline changes, new stack support, security updates |

> **Note on deployment monitoring:** This RFC does not assume DevOps monitors individual app deployments. At SleepNumber, deployment monitoring responsibility varies by org and platform. The owning team (developer, SRE, or DevOps) should be defined per app or per team as part of onboarding. The `/ship-it` workflow does not prescribe who monitors — only that the production environment gate is approved before deploy.

**What DevOps does NOT need to do**

- Create branches for developers
- Write PR descriptions
- Apply labels
- Set up initial workflow files (the skill handles first-time generation)
- Explain Git/GitHub to developers
- Manually trigger deployments

---

### 5. GitHub Actions Architecture

#### Where the workflow lives

```
repo-root/
  .github/
    workflows/
      ship-it.yml          <-- caller workflow generated by /ship-it (references the shared reusable workflow)
  .ship-it.yml             <-- DevOps team config (reviewers, environments, prerequisites)
  CODEOWNERS               <-- optional, used for reviewer fallback
  ...app code...
```

#### Workflow architecture: shared reusable workflow

Rather than generating a full standalone workflow in every repo, `/ship-it` generates a lightweight **caller workflow** that references the org's shared reusable workflow. This means:

- **One reusable workflow to maintain** — DevOps updates it once, all repos inherit changes
- **No per-repo workflow drift** — every repo calls the same build/lint/scan/deploy logic
- **Repos stay simple** — the caller workflow is a few lines, not a full pipeline definition

```
REPO (caller)                              ORG SHARED REPO (reusable workflow)
=============                              ===================================

.github/workflows/ship-it.yml             .github/workflows/ship-it-pipeline.yml
  |                                          |
  |--- uses: SleepNumberInc/                |--- build-and-validate job
  |    sleep-number-claude-code-plugins/    |    (build, lint, security scan)
  |    .github/workflows/                   |
  |    ship-it-pipeline.yml@main            |--- deploy-dev job
  |                                          |    (auto on merge to main)
  |--- with:                                |
  |      environment-dev: dev               |--- deploy-prod job
  |      environment-prod: prd       |    (requires approval)
  |      acr-login-server: ...              |
  |      aca-environment: ...               |
```

The caller workflow generated by `/ship-it` in each repo:

```yaml
name: ship-it pipeline
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  ship-it:
    uses: SleepNumberInc/sleep-number-claude-code-plugins/.github/workflows/ship-it-pipeline.yml@main
    with:
      environment-dev: dev
      environment-prod: prd
```

```
    secrets: inherit
```

The shared reusable workflow (maintained by DevOps) contains all build, lint, scan, and deploy logic. It handles stack detection, container builds, ACR push, and ACA deployment. When DevOps updates the reusable workflow, every repo that calls it picks up the changes automatically.

#### Workflow structure

The shared reusable workflow contains three jobs:

```
                    PR opened/updated
                           |
                           v
                +-----------------------+
                | build-and-validate    |
                | - Checkout            |
                | - Build               |
                | - Lint                |
                | - Security scan       |
                +-----------------------+
                           |
                    PR merged to main
                           |
                           v
                +-----------------------+
                | deploy-dev            |
                | - Checkout            |
                | - Deploy to dev env   |
                | (auto, no approval)   |
                +-----------------------+
                           |
                    Dev deploy succeeds
                           |
                           v
                +-----------------------+
                | deploy-prod           |
                | - WAIT for approval   | <-- DevOps approves via GitHub Environment UI
                | - Checkout            |
                | - Deploy to prod env  |
                +-----------------------+
```

#### Stack detection (in the shared reusable workflow)

The reusable workflow detects the project type and runs the appropriate commands:

| Project type | Detection | Build | Lint | Security |
|---|---|---|---|---|
| Node.js | `package.json` | `npm ci && npm run build --if-present` | `npm run lint --if-present` | `npm audit --audit-level=moderate` |
| Python | `requirements.txt` / `pyproject.toml` | `pip install -r requirements.txt` | `flake8 . \\\|\\\| true` | `pip-audit \\\|\\\| true` |
| Go | `go.mod` | `go build ./...` | `golangci-lint run \\\|\\\| true` | `govulncheck ./... \\\|\\\| true` |

| | | | | |
|---|---|---|---|---|
| Java (Maven) | `pom.xml` | `mvn package -DskipTests` | `mvn checkstyle:check \\\|\\\| true` | `mvn dependency-check:check \\\|\\\| true` |
| Java (Gradle) | `build.gradle` | `gradle build -x test` | `gradle check \\\|\\\| true` | `gradle dependencyCheckAnalyze \\\|\\\| true` |
| Rust | `Cargo.toml` | `cargo build` | `cargo clippy \\\|\\\| true` | `cargo audit \\\|\\\| true` |
| .NET | `*.csproj` / `*.sln` | `dotnet build` | `dotnet format --verify-no-changes \\\|\\\| true` | `dotnet list package --vulnerable \\\|\\\| true` |
| Unknown | (none of the above) | Placeholder — DevOps fills in | Placeholder | Placeholder |

**DevOps action item:** Build and maintain the shared reusable workflow at `SleepNumberInc/sleep-number-claude-code-plugins/.github/workflows/ship-it-pipeline.yml`. This is the single source of truth for all build, scan, and deploy logic.

#### GitHub Actions triggers

| Event | What runs |
|---|---|
| PR opened or updated against `main` | `build-and-validate` only |
| Push to `main` (PR merged) | `build-and-validate` → `deploy-dev` → `deploy-prod` (with approval gate) |

#### Labels created by the skill

| Label | Color | Purpose |
|---|---|---|

| | | |
|---|---|---|
| `intent:prod-ready` | Red (#e11d48) | Marks a PR as heading to production |
| `ship-it-managed` | Blue (#1d76db) | Marks a PR as created/managed by the skill |

---

### 6. The .ship-it.yml Configuration (DevOps-Owned)

This file lives in the repo root. The DevOps team creates and maintains it. Developers never touch it. The `/ship-it` skill reads it automatically.

```yaml
# .ship-it.yml — DevOps team configuration
# Drop this in any repo. /ship-it reads it automatically.

# Who gets auto-assigned to review every production PR
reviewers:
  - devops-lead
  - team-lead

# GitHub Environment names (must match Settings > Environments)
environments:
  dev: dev
  production: prd

# Shared reusable workflow reference. /ship-it generates a caller workflow
# that points here. DevOps maintains this centrally — all repos inherit changes.
# reusable_workflow: SleepNumberInc/sleep-number-claude-code-plugins/.github/workflows/ship-it-pipeline.yml@main

# Go-live checklist (shown in every production PR as checkboxes).
# These are NOT tasks for the developer to complete. They are questions
# for the developer to answer so that DevOps/platform knows what to set up.
# The developer checks the box if the answer is "yes, my app needs this."
# DevOps/platform then handles the actual setup.
prerequisites:
  - "Does this app need users to log in? (SSO / App registration)"
  - "Does this need a secure web address? (SSL certificate)"
  - "Who should have access in production? (Permissions / RBAC)"
  - "Does this need a URL people can visit? (DNS setup)"
  - "Does this need to talk to internal systems? (Network / firewall)"
  - "What type of data does this app handle? (Data classification)"
  - "Is monitoring, alerting, and logging set up? (Observability)"

# Default app description (used in PR title if none is provided)
# description: "My application"
```

#### How prerequisites are validated

Checking a box in the PR signals that the app needs a prerequisite, but does not confirm it's been set up correctly. Each prerequisite needs a validation step before production approval.

| Prerequisite | Who validates | How to validate |
|---|---|---|
| **SSO / App registration** | DevOps + Developer | Confirm the App Registration exists in Azure AD, redirect URIs are configured, and the developer can |

| | | |
|---|---|---|
| | | authenticate against it in the dev environment. Test login flow in dev before promoting to prd. |
| **SSL certificate** | DevOps | Verify cert is provisioned and bound to the custom domain. Check expiry date. Confirm HTTPS works in dev. See SSL strategy below. |
| **Permissions / RBAC** | DevOps | Review role assignments. Confirm the app's managed identity has only the permissions it needs (least privilege). |
| **DNS setup** | DevOps | Verify DNS record resolves to the correct endpoint. Test in dev first. |
| **Network / firewall** | DevOps | Confirm the app can reach required internal resources from within the ACA Environment. Test connectivity in dev. |
| **Data classification** | Developer + Security | Developer documents what data the app handles. Security team reviews if classification is Confidential or higher. |
| **Monitoring / alerting / logging** | DevOps + Developer | Confirm logs are flowing to Log Analytics (or equivalent). Verify at least one alert rule exists for app health. Test that alerts fire in dev. |

> **Production approval gate:** The production environment approver should verify that all checked prerequisites have been validated before approving the deployment. Unchecked items are assumed not applicable. Checked items without validation should block production approval.

#### SSL certificate strategy

There are two approaches to SSL for apps in the shared ACA Environment. The recommended approach eliminates per-app cert management entirely.

| Approach | How it works | Per-app effort | Cert management |
|---|---|---|---|
| **Wildcard certificate (recommended)** | Provision a single wildcard cert (e.g., `*.apps.sleepnumber.com`) and bind it to the ACA Environment. All apps use subdomains like `myapp.apps.sleepnumber.com`. | Zero — just configure the subdomain | One cert to manage, one renewal |
| **Per-app certificate** | Each app gets its own cert for its own domain. | DNS + cert provisioning per app | Multiple certs to track and renew |

**If wildcards are not feasible**, the cert process can be fully automated:

| Option | How it works |
|---|---|
| **Azure Managed Certificates** | ACA supports free managed certificates for custom domains. Azure handles provisioning and auto-renewal. No manual cert management required. Limited to domains validated via CNAME. |
| **Azure Key Vault + automation** | Store certs in Key Vault, use a Terraform module or Azure Policy to automate provisioning and binding. Certs auto-renew if using an integrated CA (e.g., DigiCert via Key Vault). |

| | |
|---|---|
| **Let's Encrypt + cert-manager** | If running on AKS, cert-manager can automate Let's Encrypt cert provisioning and renewal. Not natively supported on ACA. |

> **Recommendation:** Use a wildcard cert for the shared domain. This aligns with the "solve it once at the platform level" approach — one cert, one renewal, zero per-app overhead. Apps that need their own unique domain can use Azure Managed Certificates for automated provisioning.
>
> **Open question for DevOps:** Is a wildcard cert for `*.apps.sleepnumber.com` (or similar) viable given current security policy and cert procurement process?

#### What happens when `.ship-it.yml` is missing

The skill uses these defaults and does not block:

| Field | Default |
|---|---|
| `reviewers` | None (checks CODEOWNERS as fallback, otherwise skips) |
| `environments.dev` | `dev` |
| `environments.production` | `prd` |
| `reusable_workflow` | `SleepNumberInc/sleep-number-claude-code-plugins/.github/workflows/ship-it-pipeline.yml@main` |
| `prerequisites` | Generic checklist (SSO, SSL, permissions, DNS, network) |

---

### 7. Prerequisites

#### For the Developer (user of /ship-it)

| Requirement | How to get it | One-time? |
|---|---|---|
| Claude Code installed | `brew install claude-code` or `npm install -g @anthropic-ai/claude-code` | Yes |
| `/ship-it` plugin installed | `/plugin marketplace add SleepNumberInc/sl` | Yes |

| | | |
|---|---|---|
| | `eep-number-claude-code-plugins` then `/plugin install ship-it` | |
| GitHub CLI ( `gh` ) installed | `brew install gh` (Mac) / `sudo apt install gh` (Linux) | Yes |
| GitHub CLI authenticated | `gh auth login` | Yes |
| Git installed | Pre-installed on most systems | Yes |
| Code cloned locally | `git clone <repo-url>` | Per repo |

**That's it.** The developer does not need to know about workflows, environments, labels, or branch strategies.

**For the DevOps Team (per repo)**

| Requirement | How to do it | One-time? |
|---|---|---|
| Drop `.ship-it.yml` into the repo root | Clone the template from the repo | Yes |
| Create GitHub Environments | Repo Settings > Environments > New: `dev` and `prd` | Yes |
| Add required reviewers to `prd` environment | Repo Settings > Environments > `prd` > Required reviewers | Yes |
| Configure deployment credentials per environment | Add OIDC/secrets to each GitHub Environment (see [Section 4 — Deployment credentials]) | Yes |
| Enable branch protection on `main` | Repo Settings > Branches > Add rule: require PR, require status checks, require review | Yes |
| Optionally add `CODEOWNERS` | Create `.github/CODEOWNER` | Yes |

| | | |
|---|---|---|
| | `S` or `CODEOWNERS` in repo root | |

**For Platform/Infrastructure (one-time, org-wide)**

| Requirement | Details | One-time? |
|---|---|---|
| Claude Code available to developers | Ensure Claude Code is approved and accessible | Yes |
| GitHub Actions enabled on repos | Org-level setting | Yes |
| GitHub Environments feature available | Requires GitHub Team, Enterprise, or public repos | Yes |

**Open question for reviewers**

> **Question for DevOps:** What is the target environment structure?
>
> - `dev` → `prd` (two-tier, current default)
> - `dev` → `stg` → `prd` (three-tier)
> - Something else?
>
> The skill and workflow can support any structure. The `.ship-it.yml` config just needs to list the environments.

> **Question for DevOps:** Are you using GitHub-hosted runners (`ubuntu-latest`) or self-hosted runners?
>
> - Self-hosted, the generated workflow needs the correct `runs-on` label for private networking

---

### 8. End-to-End Workflow: Code to Production

This is the full sequence showing every system involved, from the moment a developer writes code to the moment it's live in production.

```
DEVELOPER                 /SHIP-IT SKILL              GITHUB
GITHUB ACTIONS              DEVOPS ENGINEER
=========                 ==============              ======
==============              ===============

Writes code locally
       |
       |--- runs /ship-it ------>|
                                 |
                                 |-- reads .ship-it.yml
                                 |-- detects branch, auth
                                 |-- detects project type
                                 |-- checks for blockers
                                 |
                                 |-- creates branch -------->|
```

```
                                |-- commits changes ------->|
                                |-- pushes branch --------->|
                                |                           |
                                |-- creates labels -------->|
                                |-- creates PR ------------>|--- PR opened ------>|
                                |-- assigns reviewers ----->|
                                |-- build-and-validate
                                |-- adds go-live checklist->|
                                |   (build, lint, scan)
                                |                           |
                                |<-- returns PR URL --------|
                                |
       |<-- "Done!" -------------|                          |
                                |
       |                                                   |
                                |
  (developer is done)                                      |<-- checks pass -----|
                                                           |
                                                           |
                                                           |
                                                           |--- review requested -------------->|
                                                           |
                                                           |-- reviews code
                                                           |
                                                           |-- approves PR
                                                           |<-- PR approved --------------------|
                                                           |
                                                           |
                                                           |--- PR merged --->|                  |
                                                           |   (to main)      |                  |
                                                           |
                                                           |-- deploy-dev     |
                                                           |
                                                           |   (automatic)    |
                                                           |
                                                           |                  |
                                                           |
                                                           |              validates in dev
                                                           |
                                                           |                  |
                                                           |
                                                           |<-- approval -----|
                                                           |
                                                           |                  |
                                                           |
                                                           |-- deploy-prod    |
                                                           |
                                                           |   (runs)         |
                                                           |
                                                           |                  |
                                                           |
                                                           |-- LIVE! -------->|
                                                           |
                                                           confirms healthy
                                                           |
                                                                              developer notified
                                                                              "Your app is live!"
```

**What the skill solves at each stage**

| Stage | Without /ship-it (today) | With /ship-it |
|---|---|---|
| **Branch creation** | Developer must know branching strategy, naming conventions | Skill creates it automatically with a clean short name |

| | | |
|---|---|---|
| **Repo hygiene** | Cache files, `node_modules`, `__pycache__`, build artifacts end up in commits | Skill ensures `.gitignore` covers local dev dependencies and cache files before committing (see repo cleanup) |
| **Committing & pushing** | Developer must know git add, commit, push, set upstream | Skill handles it silently |
| **PR creation** | Developer must write a description, pick labels, know who to tag | Skill generates everything from `.ship-it.yml` + commit history |
| **Pipeline setup** | Developer must write YAML, understand workflow triggers, know which checks to run | Skill generates a caller workflow that references the org's shared reusable workflow |
| **Reviewer assignment** | Developer must know who the reviewers are | Skill reads from `.ship-it.yml` or CODEOWNERS |
| **Go-live checklist** | Developer must know what infrastructure prerequisites exist | Skill includes the checklist from `.ship-it.yml` |
| **Deploy to dev** | DevOps runs manually or developer must understand pipeline | GitHub Actions runs automatically on merge |
| **Deploy to prod** | Multiple manual steps, tickets, approvals | GitHub Environment approval gate — one click |
| **Notifications** | Developer has to check GitHub manually for status updates | Teams message sent when build succeeds and status checks pass (see notifications) |
| **Handling repeat runs** | Developer creates duplicate PRs or gets confused | Skill detects existing PR, updates it or shows status |
| **Saving progress** | Developer may not know how to push safely | `/ship-it save` handles it, creates a draft PR |

#### Repo cleanup

Before committing, the `/ship-it` skill checks for and addresses common repo hygiene issues that new developers often miss:

`.gitignore` enforcement: The skill verifies a `.gitignore` file exists. If one is missing, it generates a `.gitignore` based on the detected stack before committing. If one exists, the skill checks for common omissions and warns the developer.

Files and directories that should never be committed:

| Stack | Files/directories to exclude |
|---|---|
| **Node.js** | `node_modules/`, `.npm/`, `.yarn/`, `dist/`, `.next/`, `.nuxt/` |
| **Python** | `__pycache__/`, `*.pyc`, `.venv/`, `venv/`, `.eggs/`, `*.egg-info/` |
| **Go** | `vendor/` (if not vendoring), binary outputs |
| **.NET** | `bin/`, `obj/`, `*.user`, `*.suo`, `packages/` |
| **Java** | `target/`, `build/`, `.gradle/`, `*.class` |
| **Rust** | `target/` |
| **General** | `.env`, `.env.local`, `.DS_Store`, `Thumbs.db`, `*.log`, IDE directories (`.idea/`, `.vscode/`, `.vs/`) |

**DevOps action item:** Include standardized `.gitignore` templates in repo creation templates so this check is a safety net, not the primary defense.

#### Notifications

The shared reusable workflow should send a Microsoft Teams message when the build succeeds and all status checks pass, notifying the team that the PR is ready for review or that a deployment is ready for approval.

| Event | Teams notification |
|---|---|
| PR checks all pass | "Build succeeded for **{repo}** — PR #{number} is ready for review: {PR URL}" |

| | |
|---|---|
| Deploy to dev succeeds | "**{repo}** deployed to dev — ready for validation: {dev URL if available}" |
| Deploy to prd succeeds | "**{repo}** is live in production" |
| PR checks fail | "Build failed for **{repo}** — PR #{number}: {summary of failure}" |

**Implementation:** Use the Microsoft Teams Incoming Webhook or the Teams Workflow integration in the shared reusable workflow. The webhook URL should be stored as a GitHub org-level secret (e.g., `TEAMS_WEBHOOK_URL`) so all repos inherit it.

**DevOps action item:** Add Teams notification steps to the shared reusable workflow. This also resolves **Q5 (Notifications)** from the open questions.

---

### 9. Security Considerations

| Concern | How it's addressed |
|---|---|
| **Secrets in code** | The skill commits with `git add -A`. `.gitignore` must be properly configured. DevOps should ensure `.gitignore` templates are in place for each repo. |
| **Branch protection bypass** | The skill creates PRs — it never pushes directly to `main`. Branch protection rules remain fully enforced. |
| **Production deployment without approval** | The `prd` GitHub Environment requires reviewer approval. The skill cannot bypass this. |
| **Reviewer manipulation** | Reviewers are defined in `.ship-it.yml` which is committed to the repo and version-controlled. Changes require a PR. |
| **GitHub token scope** | The skill uses the developer's existing `gh` CLI authentication. No additional tokens are stored or created. |
| **Workflow tampering** | If a workflow already exists, the skill does NOT overwrite it. DevOps-managed workflows are respected. |
| **Label manipulation** | Labels are informational. Deployment gates are enforced by |

| | |
|---|---|
| | GitHub Environments, not labels. |

#### Recommendation

DevOps should add `.ship-it.yml` to the repo's protected files (require review for changes) to prevent developers from modifying reviewer lists or prerequisites.

---

### 10. Recommendation: Containers as the Standardized Deployment Unit

To simplify the deployment process for both developers and DevOps, we recommend **containerization as the standard packaging model** for all apps deployed through this workflow.

#### Why containers

Requiring a Dockerfile as the standard deployment artifact creates a single, repeatable path from code to production:

- **For developers:** Write code, add a Dockerfile, run `/ship-it`. No need to understand App Service Plans, AKS clusters, networking, or RBAC. The container runs the same locally and in production.
- **For DevOps:** One deployment pattern to support instead of many. GHA workflows follow a consistent flow: build image → push to ACR → deploy to shared platform. Deployer credentials, registry access, and RBAC are centralized, not per-app.
- **For networking:** This is the biggest win. A shared container platform (Azure Container Apps Environment or AKS) has networking built in at the platform level. Individual apps inherit networking from the shared environment — no per-app VNet integration, no per-app firewall rules, no per-app subnet provisioning. The only exceptions are apps requiring external DNS entries or specific firewall openings.

#### Recommended platform: Azure Container Apps with your own VNet

Azure Container Apps (ACA) is the recommended container platform for this workflow. ACA is a managed container service — Microsoft handles the underlying Kubernetes infrastructure, so DevOps does not need to manage clusters directly.

**The critical networking decision happens once, at environment creation time.** ACA offers two networking modes:

| Option | What you get | Can reach internal systems? | Can change later? |
|---|---|---|---|
| **Default (automatically generated VNet)** | Microsoft creates and manages the VNet. Limited control. | No | **No — permanent** |
| **Existing VNet (you provide your own)** | You provide your own VNet and a dedicated subnet. Full networking capabilities. | Yes | **No — permanent** |

**We recommend creating the ACA Environment with an existing VNet that you provide.** This is a one-time, irreversible decision. If the environment is created with the automatically generated VNet, apps cannot reach internal resources (databases, APIs, corporate systems), and you would need to create an entirely new environment to fix it.

#### What using your own VNet gives you

- Integration with Application Gateway and Network Security Groups (NSGs)
- Communication with resources behind private endpoints
- Connectivity to internal systems via VNet peering or ExpressRoute
- Option to make the environment **internal-only** ( `--internal-only true` ) — no public internet access, apps only reachable from within the VNet

#### Subnet requirements (workload profiles environment)

The ACA Environment requires a **dedicated subnet** — it cannot be shared with other services.

| Requirement | Value |
|---|---|
| Minimum subnet size | `/27` |
| Subnet delegation | Must delegate to `Microsoft.App/environments` |
| Reserved IPs for infrastructure | 14 (12 for ACA + 2 for Azure) |
| Subnet resizable after creation? | **No** |

Plan subnet size based on expected scale:

| Subnet size | Available IPs | Max nodes (Dedicated profile) | Max replicas (Consumption profile) |
|---|---|---|---|
| /23 | 498 | 249 | 2,490 |
| /24 | 242 | 121 | 1,210 |
| /25 | 114 | 57 | 570 |
| /26 | 50 | 25 | 250 |
| /27 | 18 | 9 | 90 |

> **Recommendation:** Start with at least a `/24` subnet to allow room for growth. Subnet size cannot be changed after creation.

#### Subnet address range restrictions

Subnet ranges cannot overlap with these ranges reserved by the underlying AKS infrastructure:

- `169.254.0.0/16`
- `172.30.0.0/16`
- `172.31.0.0/16`

- `192.0.2.0/24`

Additionally, the workload profiles environment reserves:

- `100.100.0.0/17`
- `100.100.128.0/19`
- `100.100.160.0/19`
- `100.100.192.0/19`

#### Shared platform model

The recommended infrastructure model:

| Layer | Provisioned by | Frequency | What it includes |
|---|---|---|---|
| **Azure subscription & resource group** | Platform team | Once | Shared across all apps |
| **Your own VNet & dedicated subnet** | Platform team | Once | Networking boundary, peered to corporate network, subnet delegated to `Microsoft.App/environments` |
| **ACA Environment** (with your own VNet) | Platform team | Once | Compute, ingress, networking — all apps inherit from here |
| **Container registry** (ACR) | Platform team | Once | Shared image storage, deployer identity has push/pull |
| **Deployer identity** (App Registration + RBAC) | Platform team | Once | Shared credentials used by GHA to push images and deploy |
| **Individual app deployment** | GHA (triggered by PR merge) | Per deploy | Deploys into the existing shared environment |

This model means:

- **No per-app infrastructure provisioning** — apps deploy into what already exists
- **No per-app networking setup** — every app inherits networking from the ACA Environment's virtual network
- **No per-app credential management** — shared deployer identity with scoped RBAC
- **Networking is only a per-app concern** when an app needs a public DNS entry or a specific firewall rule to reach an external system

#### Known limitations of a shared ACA Environment

A shared Container Apps Environment solves the 80% case, but there are caveats that affect certain apps. These were identified during DevOps review and should be accounted for in platform design.

##### 1. Secrets management and isolation

Apps in a shared ACA Environment can each have their own managed identity and Azure Key Vault references, but there is no enforced secrets isolation boundary between apps within the same environment. A misconfigured managed identity could potentially access another app's secrets.

**Mitigation:** Each app should use its own managed identity with RBAC scoped to only its own Key Vault. DevOps should enforce this as part of the app onboarding process. Apps with strict regulatory or compliance requirements around secrets isolation should use a separate ACA Environment.

##### 2. No network isolation within the environment

All apps in the same Container Apps Environment can freely communicate with each other over the internal network. There are no NSGs or network boundaries between apps within an environment.

**Mitigation:** For the majority of apps, this is acceptable — apps in the same environment are typically in the same trust boundary. **High-security apps that require network isolation from other apps must be deployed to a separate ACA Environment** with its own VNet and subnet. The Terraform IaC in the `ACA_CVNet_tf` project supports creating multiple environments for this purpose.

##### 3. DNS and TLS certificates per production app

Each production app with a custom domain requires its own DNS entry and TLS certificate. This is a per-app setup step that cannot be avoided.

**Mitigation:** Use a wildcard certificate (e.g., `*.apps.sleepnumber.com`) and a wildcard DNS entry to cover all apps under a single domain. This reduces per-app DNS/cert overhead to zero for apps that can share the wildcard domain. Apps requiring their own unique domain still need individual setup.

#### What this means for /ship-it

The skill itself remains unchanged — it is scope-bounded to getting code into a PR. But this recommendation shapes what the GHA workflow looks like on the DevOps side:

```
PR merged → GHA workflow:
  1. docker build → tag with commit SHA
  2. docker push → ACR
  3. az containerapp update → deploy to shared ACA Environment
```

DevOps configures this once per repo (or via a shared reusable action), and every app follows the same pattern.

GHA deploy workflows and infrastructure provisioning should adopt SleepNumberInc's shared Terraform modules to ensure consistency across all apps deployed into the ACA platform. The shared platform itself (VNet, ACA

Environment, ACR, deployer identity) is provisioned using the `ACA_CVNet_tf` Terraform project, and per-app deployment should reference the same module patterns rather than hand-rolling infrastructure per repo.

#### Prerequisite: Phase 0 — Shared platform setup

Before pilot repos can use this flow, the shared container platform must be in place. This should be treated as **Phase 0** of the rollout plan:

- [ ] Create your own VNet with a dedicated subnet (minimum `/24` recommended)
- [ ] Delegate the subnet to `Microsoft.App/environments`
- [ ] Peer the VNet to the corporate network (if apps need internal connectivity)
- [ ] Create the ACA Environment with your existing VNet ( `--internal-only` if no public access needed)
- [ ] Create an Azure Container Registry (ACR)
- [ ] Create a deployer App Registration with RBAC scoped to the resource group, ACA Environment, and ACR
- [ ] Configure GHA secrets/OIDC for the deployer identity

> **Reference:** [Configuring virtual networks for Azure Container Apps environments]

---

### 11. Open Questions for Reviewers

These questions need input from DevOps and Platform engineers before finalizing.

#### Environment Structure

> **Q1:** ~~What is the target environment tier structure?~~
>
> **RESOLVED:** Two-tier: `dev` → `prd`. Confirmed by DevOps. All environment references in this RFC have been updated accordingly.

#### Runners

> **Q2:** ~~What GitHub Actions runner type should be used?~~
>
> **RESOLVED:** Self-hosted runners. Required for private networking — GitHub-hosted runners cannot reach resources inside the private VNet. The shared reusable workflow must use self-hosted runner labels in `runs-on`.
>
> **DevOps action item:** Provide the self-hosted runner label(s) to use in the shared reusable workflow (e.g., `runs-on: [self-hosted, linux]`). Self-hosted runners must be deployed inside the VNet (or have network access to the ACA Environment and ACR) to perform deployments.

#### Deployment Commands

> **Q3:** What are the actual deployment commands for your apps?
>
> The generated workflow uses placeholder deploy steps. DevOps needs to fill in the real commands. Examples:
>
> - Kubernetes: `kubectl apply -f k8s/` or `helm upgrade`
> - AWS: `aws ecs update-service` or `cdk deploy`
> - Azure: `az webapp deploy`
> - Container registry: `docker push` + deploy trigger
>
> Should these be standardized in a shared action (e.g., `sleepnumberinc/deploy-action`)?

#### Workflow Ownership

> **Q4:** ~~Should DevOps pre-configure workflows in every repo, or let /ship-it generate them on first run?~~
>
> **RESOLVED:** Use a shared reusable workflow maintained centrally by DevOps. The `/ship-it` skill generates a lightweight caller workflow in each repo that references the shared reusable workflow. DevOps updates the reusable workflow once; all repos inherit changes automatically. No per-repo workflow drift, no disparate workflows to keep in sync.
>
> See [Section 5 — Workflow architecture]() for details.

#### Notifications

> **Q5:** ~~How should the developer be notified when their app is live?~~
>
> **RESOLVED:** Microsoft Teams notifications via the shared reusable workflow. Teams messages are sent when builds pass (ready for review), when dev deploys succeed (ready for validation), and when production deploys succeed (app is live). See [Section 8 — Notifications]() for details.

#### Additional Prerequisites

> **Q6:** ~~Are there additional go-live prerequisites beyond the defaults?~~
>
> **RESOLVED:** Add data classification and monitoring/alerting/logging to the default prerequisites. Updated checklist:
>
> - SSO / App registration
> - SSL certificate
> - Permissions / RBAC
> - DNS setup
> - Network / firewall
> - **Data classification** (what type of data does this app handle?)
> - **Monitoring / alerting / logging** (is observability configured?)

---

### 12. Rollout Plan

#### Phase 0 — Shared Platform Setup (prerequisite — before pilot)

> **Owner:** Platform / DevOps team
> **Blocker:** Phases 1-3 cannot begin until Phase 0 is complete. Apps need a shared container platform to deploy into.

**Networking & VNet**

- [ ] Create your own VNet with a dedicated subnet (minimum `/24` recommended — see Section 10 subnet sizing)
- [ ] Delegate the subnet to `Microsoft.App/environments`
- [ ] Peer the VNet to the corporate network (if apps need internal connectivity)
- [ ] Configure NSGs on the subnet as needed

**Azure Container Apps Environment**

- [ ] Create the ACA Environment with your existing VNet
- [ ] Set accessibility level: external or internal-only (`--internal-only true` if no public access needed)
- [ ] Verify apps deployed into the environment can reach required internal resources

**Container Registry & Identity**

- [ ] Create an Azure Container Registry (ACR)
- [ ] Create a deployer App Registration with RBAC scoped to the resource group, ACA Environment, and ACR
- [ ] Configure OIDC federation on the App Registration for the GitHub repo(s)
- [ ] Add deployment credentials to GitHub Environment secrets (see Section 4 — Deployment credentials by platform)
- [ ] Verify GHA can authenticate, push images to ACR, and deploy to ACA
- [ ] If targeting additional platforms (AWS, k3s), configure credentials per the platform table in Section 4

**Self-Hosted Runners**

- [ ] Deploy self-hosted GHA runners inside the VNet (or with network access to the ACA Environment and ACR)
- [ ] Register runners with the GitHub org/repo and assign runner labels
- [ ] Verify runners can reach ACR (image push/pull) and ACA (deploy commands)

**Validation**

- [ ] Deploy a test container app into the environment using a GHA workflow on the self-hosted runner to confirm the full pipeline end-to-end
- [ ] Document the shared resource names (resource group, ACA Environment, ACR, deployer identity, runner labels) for use in `.ship-it.yml` and GHA workflows

> **Important:** The VNet and ACA Environment networking configuration is **permanent and cannot be changed after creation**. If the environment is created with the default automatically generated VNet instead of your own VNet, it must be recreated from scratch. Get this right the first time.

**Phase 1 — Pilot (Weeks 1-2)**

- [ ] DevOps reviews this RFC and answers open questions
- [ ] Select 1-2 pilot repos
- [ ] Ensure pilot repos have a Dockerfile (or add one)
- [ ] Drop `.ship-it.yml` into pilot repos
- [ ] Configure GitHub Environments (`dev`, `prd`) on pilot repos
- [ ] Fill in real deploy commands in workflow (docker build → ACR push → `az containerapp update`)
- [ ] 2-3 developers test `/ship-it` and `/ship-it save` on pilot repos
- [ ] Collect feedback, iterate on skill

**Phase 2 — DevOps Adoption (Weeks 3-4)**

- [ ] Standardize `.ship-it.yml` template for the org
- [ ] Create a shared reusable GHA action for build → push → deploy to ACA
- [ ] Add `.ship-it.yml` and Dockerfile template to repo creation templates
- [ ] Document any org-specific workflow customizations
- [ ] Train DevOps team on the review/approval flow

**Phase 3 — Broad Rollout (Weeks 5+)**

- [ ] Publish `/ship-it` to the org's Claude Code marketplace
- [ ] Add `/ship-it` to developer onboarding guide
- [ ] Roll out to remaining repos
- [ ] Monitor adoption and iterate

---

### 13. How to Review / Comment

This RFC is open for comment. Please provide feedback via:

1. **GitHub Issues** on https://github.com/SleepNumberInc/sleep-number-claude-code-plugins Connect your Github account — file an issue with the label `rfc-feedback`

2. **PR comments** — if you want to suggest specific changes to the skill or config, open a PR against the repo
3. **Inline comments** — if reviewing this document in a PR, use GitHub's inline comment feature

**What we need from you**

| Role | What to review |
| --- | --- |
| **DevOps Engineers** | Environment structure (Q1), runner type (Q2), deploy commands (Q3), workflow ownership (Q4), additional prerequisites (Q6) |
| **Platform/Infra Engineers** | Runner type (Q2), security considerations (Section 9), GitHub Environment configuration |

**Timeline**

| Milestone | Date |
| --- | --- |
| RFC published | 2026-03-09 |
| Comment period closes | TBD (suggest 2 weeks) |
| Pilot begins | TBD (after open questions resolved) |

---

*This RFC was generated as part of the /ship-it project. See the full source at https://github.com/SleepNumberInc/sleep-number-claude-code-plugins* Connect your Github account